



CIRCUITOS DIGITALES Y MICROCONTROLADORES 2025

Facultad de Ingeniería
UNLP

Conjunto de Instrucciones AVR

Ing. José Juárez

Conjunto de Instrucciones

- El conjunto de Instrucciones que un MCU puede ejecutar esta definido por el fabricante y es un único para cada familia o modelo de MCU.
- Ver la documentación completa:
 - [Documento: AVR Instrucción Set](#)
- Programación en lenguaje assembler o ensamblador:
 - Desarrollado por el fabricante
 - Bajo nivel (instrucciones en lenguaje máquina)
 - No portable a otros fabricantes
 - Máx rendimiento y aprovechamiento de recursos (velocidad de ejecución y consumo de memoria)

Conjunto de Instrucciones

- Tipos de Instrucciones:
 - Aritméticas y lógicas
 - Bifurcaciones
 - Transferencia de datos
 - Operaciones de bits y bit test
 - Especiales

Table 2-2: ALU Instructions Using Two GPRs

Instruction		
ADD	Rd, Rr	ADD Rd and Rr
ADC	Rd, Rr	ADD Rd and Rr with Carry
AND	Rd, Rr	AND Rd with Rr
EOR	Rd, Rr	Exclusive OR Rd with Rr
OR	Rd, Rr	OR Rd with Rr
SBC	Rd, Rr	Subtract Rr from Rd with carry
SUB	Rd, Rr	Subtract Rr from Rd without carry

Instruction		
CLR	Rd	Clear Register Rd
INC	Rd	Increment Rd
DEC	Rd	Decrement Rd
COM	Rd	One's Complement Rd
NEG	Rd	Negative (two's complement) Rd
ROL	Rd	Rotate left Rd through carry
ROR	Rd	Rotate right Rd through carry
LSL	Rd	Logical Shift Left Rd
LSR	Rd	Logical Shift Right Rd
ASR	Rd	Arithmetic Shift Right Rd
SWAP	Rd	Swap nibbles in Rd

Set de Instrucciones

- LDI “carga inmediata”, solo con R16 a R31

```
LDI R20,0x25           ;load R20 with 0x25 (R20 = 0x25)
```

- ADD

```
LDI R16,0x25           ;load 0x25 into R16
LDI R17,0x34           ;load 0x34 into R17
ADD R16,R17            ;add value R17 to R16 (R16 = R16 + R17)
```

- LDS “carga desde RAM”

```
LDS R0, 0x300          ;R0 = the contents of location 0x300
LDS R1, 0x302          ;R1 = the contents of location 0x302
ADD R1, R0              ;add R0 to R1
```

- STS “almacena en RAM”

```
LDS R30, 0x220         ;load R30 with the contents of location 0x220
LDS R31, 0x221         ;load R31 with the contents of location 0x221
ADD R31, R30           ;add R30 to R31
STS 0x221, R31         ;store R31 to data space location 0x221
```

Set de Instrucciones

- IN

```
IN    R1,PIND      ;load R1 with PIND
IN    R2,PINB      ;load R2 with PINB
ADD   R1, R2        ;R1 = R1 + R2
STS   0x300, R1     ;store R1 to data space location $300
```

- OUT

```
IN     R0, PIND     ;load R20 with the contents of I/O reg PIND
OUT    PORTA, R0    ;out R20 to PORTA
```

- MOV

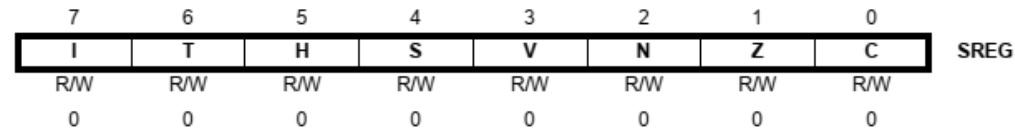
```
MOV    R10,R20      ;R10 = R20
```

- INC

```
LDS    R20, 0x430    ;R20 = contents of location 0x430
INC     R20           ;R20 = R20 + 1
STS     0x430, R20    ;store R20 to location 0x430
```

Set de Instrucciones

- Bifurcaciones y lazos



- BRNE: Branch if not equal ($Z \neq 1$)

```
LDI R16, 10      ;R16 = 10 (decimal) for counter
LDI R20, 0       ;R20 = 0
LDI R21, 3       ;R21 = 3
AGAIN:ADD R20, R21 ;add 03 to R20 (R20 = sum)
DEC R16          ;decrement R16 (counter)
BRNE AGAIN       ;repeat until COUNT = 0
OUT PORTB,R20    ;send sum to PORTB
```

¿Cuál es el valor transferido a PORTB? ¿Cuánto tiempo tardó en ejecutarse?

- Ejemplo de bucles de retardo

```
LDI R16, 0x55    ;R16 = 0x55
OUT PORTB, R16   ;PORTB = 0x55
LDI R20, 10      ;load 10 into R20 (outer loop count)
LOP_1:LDI R21, 70 ;load 70 into R21 (inner loop count)
LOP_2:COM R16     ;complement R16
OUT PORTB, R16   ;load PORTB SFR with the complemented value
DEC R21          ;dec R21 (inner loop)
BRNE LOP_2       ;repeat it 70 times
DEC R20          ;dec R20 (outer loop)
BRNE LOP_1       ;repeat it 10 times
```

Set de Instrucciones

- BREQ: branch if equal (Z=1)

```
OVER: IN    R20, PINB    ;read PINB and put it in R20
      TST   R20          ;set the flags according to R20
      BREQ  OVER         ;jump if R20 is zero
```

- Otros saltos condicionales

**Table 3-1: AVR Conditional
Branch (Jump) Instructions**

Instruction	Action
BRLO	Branch if C = 1
BRSB	Branch if C = 0
BREQ	Branch if Z = 1
BRNE	Branch if Z = 0
BRMI	Branch if N = 1
BRPL	Branch if N = 0
BRVS	Branch if V = 1
BRVC	Branch if V = 0

...

- Saltos Incondicionales: JMP (Jump hasta 64k),
RJMP (Relative jump, hasta 2k)
IJMP (Indirect Jump, usando reg. Z)

Formato de Instrucción AVR

- Veamos algunos ejemplos de formato de instrucción:

CLEAR REGISTER

Operation:
(i) $Rd \leftarrow Rd \oplus Rd$

Syntax: (i) `CLR Rd`
Operands: $0 \leq d \leq 31$

Program Counter:
 $PC \leftarrow PC + 1$

16-bit Opcode: (see EOR Rd,Rd)

0010	01dd	dddd	dddd
------	------	------	------

Status Register (SREG) and Boolean Formula:

I	T	H	S	V	N	Z	C
-	-	-	0	0	0	1	-

Example:

```
clr    r18      ; clear r18
loop:  inc    r18      ; increase r18
...
cpi    r18,$50   ; Compare r18 to $50
brne   loop
```

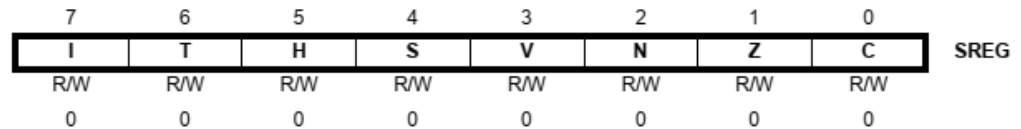
Words: 1 (2 bytes)

Cycles: 1

- Código de operación de 6 bits
- Referencia al operando de 10 bits (con 5 alcanzaría)

Formato de Instrucción AVR

• Ejemplo: ADD with Carry



Operation:
(i) $Rd \leftarrow Rd + Rr + C$

Syntax:
(i) `ADC Rd,Rr`

Operands:
 $0 \leq d \leq 31, 0 \leq r \leq 31$

Program Counter:
 $PC \leftarrow PC + 1$

16-bit Opcode:

0001	11rd	dddd	rrrr
------	------	------	------

Example:

- Código de operación de 6 bits
- Referencias a los operandos 5 bits c/u

```

; Add R1:R0 to R3:R2
add  r2,r0    ; Add low byte
adc  r3,r1    ; Add with carry high byte
    
```

Words: 1 (2 bytes)

Cycles: 1

Formato de Instrucción AVR

- Ejemplo: ADD with Carry

Status Register (SREG) Boolean Formula:

I	T	H	S	V	N	Z	C
-	-	$\Leftrightarrow$	$\Leftrightarrow$	$\Leftrightarrow$	$\Leftrightarrow$	$\Leftrightarrow$	$\Leftrightarrow$

H: $Rd3 \bullet Rr3 + Rr3 \bullet \overline{R3} + \overline{R3} \bullet Rd3$
Set if there was a carry from bit 3; cleared otherwise

S: $N \oplus V$, For signed tests.

V: $Rd7 \bullet Rr7 \bullet \overline{R7} + \overline{Rd7} \bullet \overline{Rr7} \bullet R7$
Set if two's complement overflow resulted from the operation; cleared otherwise.

N: $R7$
Set if MSB of the result is set; cleared otherwise.

Z: $\overline{R7} \bullet \overline{R6} \bullet \overline{R5} \bullet \overline{R4} \bullet \overline{R3} \bullet \overline{R2} \bullet \overline{R1} \bullet \overline{R0}$
Set if the result is \$00; cleared otherwise.

C: $Rd7 \bullet Rr7 + Rr7 \bullet \overline{R7} + \overline{R7} \bullet Rd7$
Set if there was carry from the MSB of the result; cleared otherwise.

R (Result) equals Rd after the operation.

Formato de Instrucción AVR

- Ejemplo: Instrucción IN (transferencia de registro I/O a registros CPU)

Operation:			
(i)	$Rd \leftarrow I/O(A)$		
	Syntax:	Operands:	Program Counter:
(i)	IN Rd,A	$0 \leq d \leq 31, 0 \leq A \leq 63$	$PC \leftarrow PC + 1$

16-bit Opcode:

1011	0AA <i>d</i>	<i>dddd</i>	<i>AAAA</i>
------	--------------	-------------	-------------

Status Register (SREG) and Boolean Formula:

I	T	H	S	V	N	Z	C
-	-	-	-	-	-	-	-

Example:

```

in      r25,$16    ; Read Port B
cpi     r25,4       ; Compare read value to constant
breq    exit        ; Branch if r25=4
...
exit:   nop          ; Branch destination (do nothing)
    
```

Words: 1 (2 bytes)

Cycles: 1

Formato de Instrucción AVR

- Ejemplo: Instrucción LDS (carga directa desde RAM a registros)

Operation:
(i) $Rd \leftarrow (k)$

Syntax:
(i) LDS Rd,k

Operands:
 $0 \leq d \leq 31, 0 \leq k \leq 65535$

Program Counter:
 $PC \leftarrow PC + 2$

32-bit Opcode:

1001	000d	dddd	0000
kkkk	kkkk	kkkk	kkkk

Example:

```
lds  r2,$FF00    ; Load r2 with the contents of data space location $FF00
add  r2,r1        ; add r1 to r2
sts  $FF00,r2     ; Write back
```

Words: 2 (4 bytes)

Cycles: 2

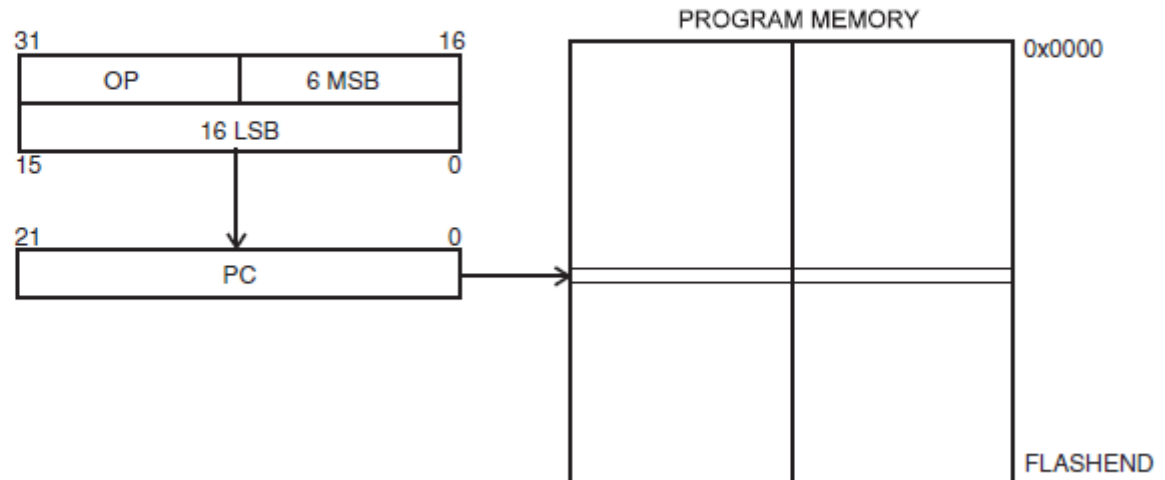
Formato de Instrucción AVR

- Ejemplo: Instrucción JMP (jump)

(i)	Syntax: JMP k	Operands: $0 \leq k < 4M$	Program Counter: $PC \leftarrow k$	Stack: Unchanged
	32-bit Opcode:			
1001	010k	kkkk	110k	Words: 2 (4 bytes) Cycles: 3
kkkk	kkkk	kkkk	kkkk	

Direct Program Addressing, JMP and CALL

Figure 11. Direct Program Memory Addressing



Modos de direccionamiento

- Veremos ahora direccionamiento a memoria de datos (SRAM, Registros CPU y Registros I/O)
- Posibilidades:
 - El dato está contenido en la instrucción
 - El dato está en un registro Rx con $0 \leq x \leq 31$
 - El número de registro (x) está en la instrucción
 - El dato está en memoria SRAM o espacio I/O
 - La dirección del dato está en instrucción
 - La dirección está en un registro Rx con $0 \leq x \leq 31$
 - El número de registro (x) está en la instrucción
 - La dirección se calcula con un registro Rx y un desplazamiento (offset)
 - El número de registro (x) está en la instrucción
 - El desplazamiento está en la instrucción

Modos de direccionamiento

Direccionamiento Inmediato (LDI)

- El dato está en la instrucción
- Ejemplo LDI o carga inmediata
 - LDI Rd,K ; k es un número de 8 bits
 - LDI r15,\$FF; cargar el registro r15 con el valor \$FF

Operation:
(i) $Rd \leftarrow K$

Syntax: (i) LDI Rd,K
Operands: $16 \leq d \leq 31, 0 \leq K \leq 255$

Program Counter:
 $PC \leftarrow PC + 1$

16-bit Opcode:

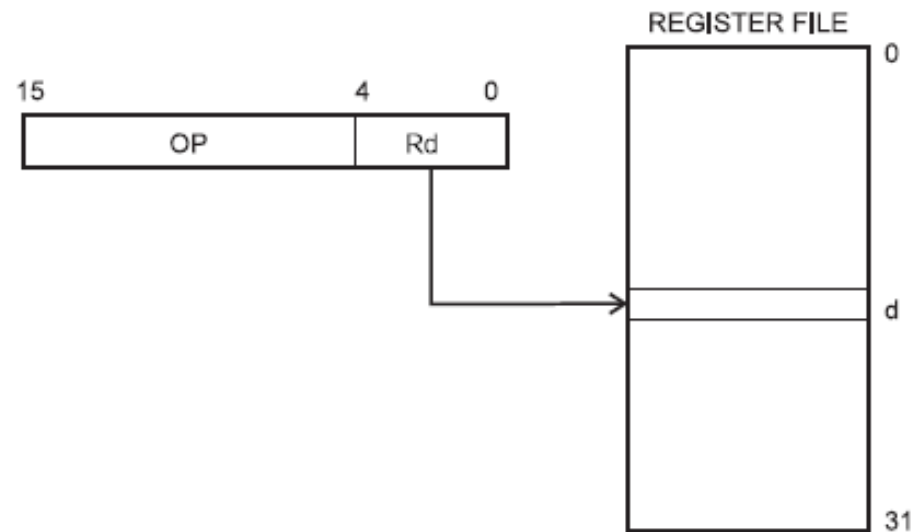
1110	KKKK	dddd	KKKK
------	------	------	------

Modos de direccionamiento

Directo a Registros

- Un único registro

Figure 1. Direct Single Register Addressing



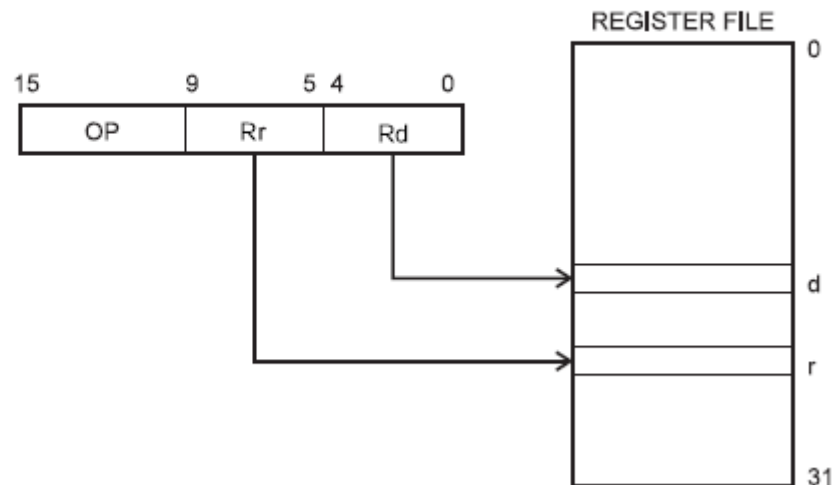
- Ejemplo: CLR Rd

Modos de direccionamiento

Directo a Registros

- Dos Registros

Figure 2. Direct Register Addressing, Two Registers

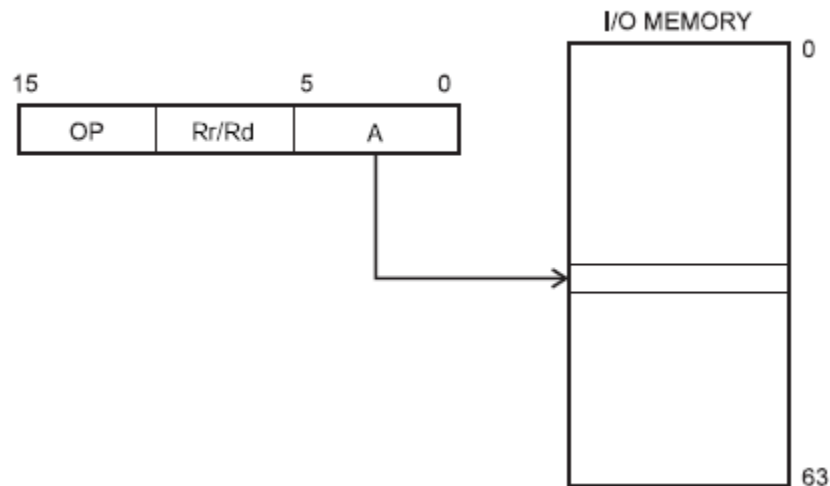


- Operandos contenidos en registros Rr y Rd.
- El resultado se almacena en Rd
- Ej: ADD Rd,Rr

Modos de direccionamiento

Directo a Registros I/O

Figure 3. I/O Direct Addressing

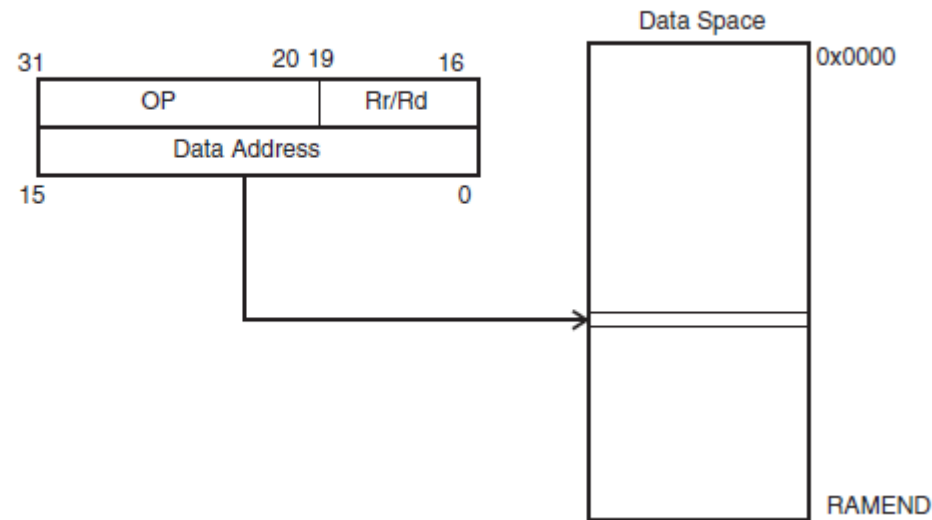


- La dirección del operando esta contenida en los últimos 6 bits
- Ejemplo: `IN r25,PORTB`

Modos de direccionamiento

Directo a memoria de datos (REG+SRAM)

Figure 4. Direct Data Addressing

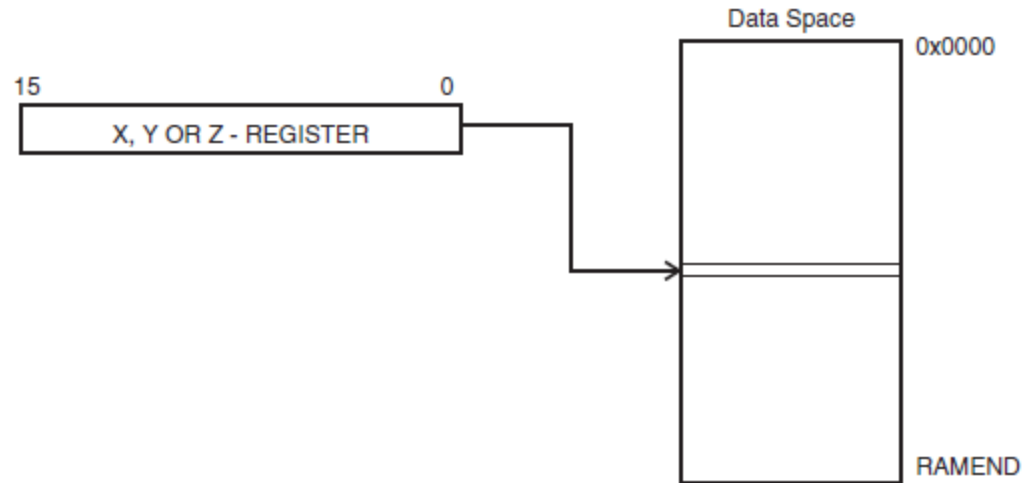


- Data Address es la dirección (16 bits) donde se encuentra el dato.
- Ejemplos:
 - LDS Rd,k ; Carga directa desde RAM
 - STS k,Rr ; Almacenamiento directo en RAM

Modos de direccionamiento

Indirecto a memoria de datos

Figure 6. Data Indirect Addressing

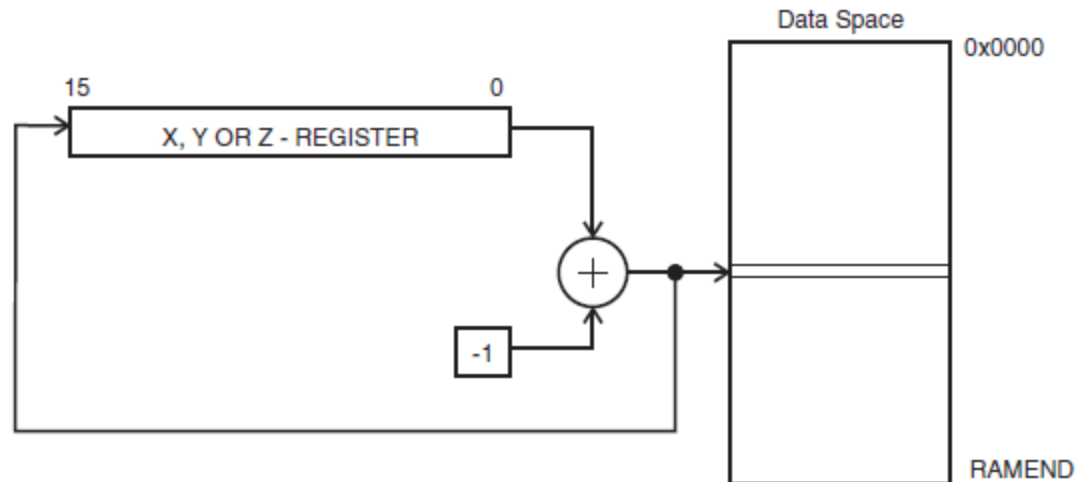


- La dirección del dato está contenida en alguno de los Registros X,Y o Z
 - LD Rd,X ; $Rd \leftarrow (X)$
 - ST Y,Rr ; $(Y) \leftarrow Rr$

Modos de direccionamiento

Indirecto con Pre-decremento

Figure 7. Data Indirect Addressing with Pre-decrement

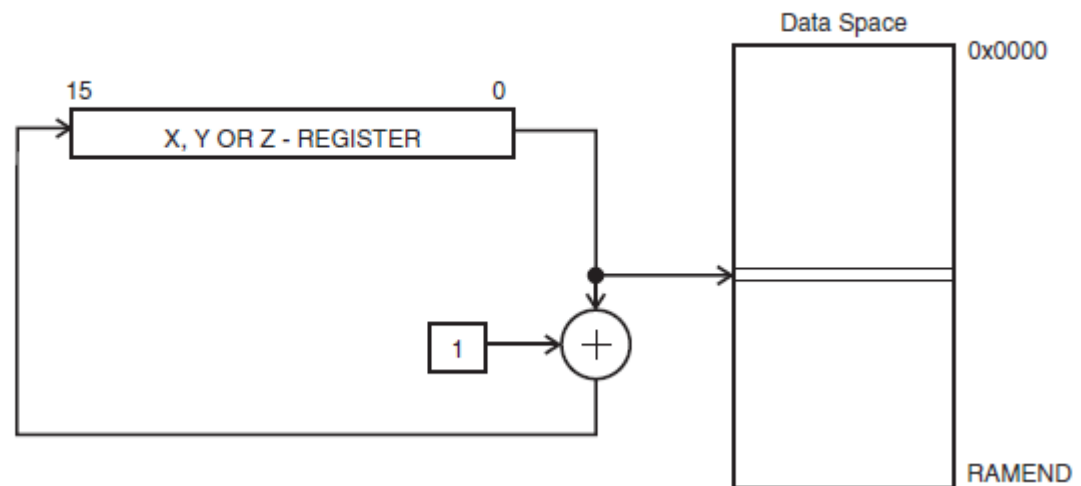


- Ejemplo:
 - LD r16, -X
 - ST -Z, R15

Modos de direccionamiento

Indirecto con pos-incremento

Figure 8. Data Indirect Addressing with Post-increment

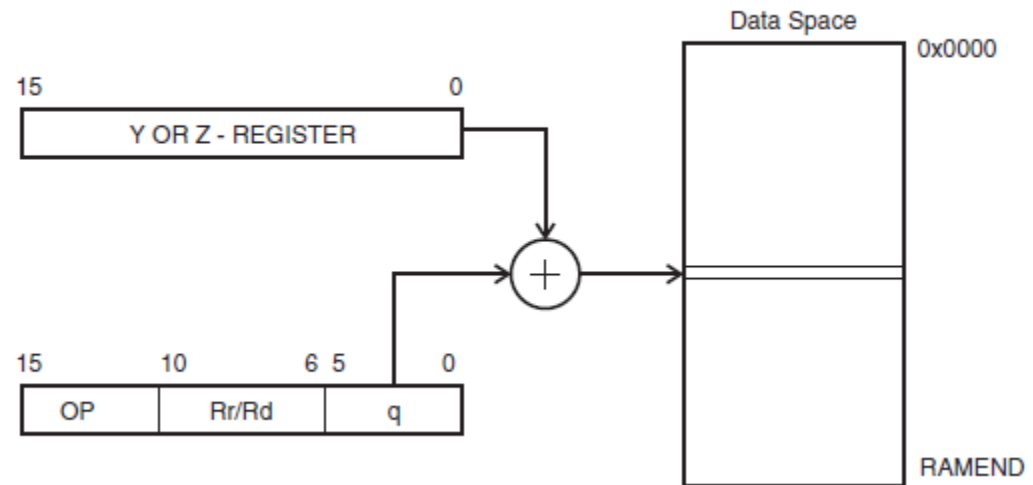


- Ejemplo:
 - LD r16,X+
 - ST Z+,r15

Modos de direccionamiento

Indirecto con desplazamiento

Figure 5. Data Indirect with Displacement



- La dirección del dato se obtiene sumando un desplazamiento a los punteros Y o Z
- LDD Rd,Z+q; $Rd \leftarrow (Z+q)$
 - LDD r16,Y+\$10
- STD Z+q,Rr ; $(Z+q) \leftarrow Rr$
 - STD Z+\$20, r15

Resumen: Modos de Direcccionamiento

- Direcccionamiento a memoria de datos (REG+SRAM)
 - Inmediato
 - Directo a 1 o 2 registros CPU
 - Directo a registros I/O
 - Directo a RAM
 - Indirecto a RAM
 - Indirecto a RAM con desplazamiento
 - Indirecto a RAM con pre decremento
 - Indirecto a RAM con pos incremento
- Direcccionamiento a memoria de programa (FLASH)
 - Directo
 - Indirecto
 - Relativo
 - Direcccionamiento a constante con y sin pos incremento

Ejemplo para probar Microchip Studio

```
;
; toggleProject.asm
;
        LDI R16,0xFF
        OUT DDRB,R16

L1:     OUT PORTB,R16
        LDI R20,0
        OUT PORTB,R20
        RJMP L1
```

Tarea:

Ejecutar en el simulador y verificar el código de máquina generado.

Ver:

[AsmProgrammingInAtmelStudio7.pdf](#)